- `config.active_support.use_standard_json_time_format` enables or disables serializing dates to ISO 8601 format. Defaults to `true`.
- `config.active_support.time_precision` sets the precision of JSON encoded time values. Defaults to `3`.
- `config.active_support.halt_callback_chains_on_return_false` specifies whether ActiveRecord, ActiveModel and ActiveModel::Validations callback chains can be halted by returning `false` in a 'before' callback. Defaults to `true`.
- `ActiveSupport::Logger.silencer` is set to `false` to disable the ability to silence logging in a block. The default is `true`.
- `ActiveSupport::Cache::Store.logger` specifies the logger to use within cache store operations.
- `ActiveSupport::Deprecation.behavior` alternative setter to `config.active_support.deprecation` which configures the behavior of deprecation warnings for Rails.
- `ActiveSupport::Deprecation.silence` takes a block in which all deprecation warnings are silenced.
- `ActiveSupport::Deprecation.silenced` sets whether or not to display deprecation warnings.

3.12 Configuring a Database

Just about every Rails application will interact with a database. You can connect to the database by setting an environment variable `ENV['DATABASE_URL']` or by using a configuration file called `config/database.yml`.

Using the `config/database.yml` file you can specify all the information needed to access your database:

```
development:
  adapter: postgresql
  database: blog_development
  pool: 5
```

This will connect to the database named `blog_development` using the `postgresql` adapter. This same information can be stored in a URL and provided via an environment variable like this:

```
> puts ENV['DATABASE_URL']
postgresql://localhost/blog_development?pool=5
```

The `config/database.yml` file contains sections for three different environments in which Rails can run by default:

- The `development` environment is used on your development/local computer as you interact manually with the application.
- The `test` environment is used when running automated tests.
- The `production` environment is used when you deploy your application for the world to use.

If you wish, you can manually specify a URL inside of your `config/database.yml`

```
development:
  url: postgresql://localhost/blog_development?pool=5
```

The `config/database.yml` file can contain ERB tags `<%= %>`. Anything in the tags will be evaluated as Ruby code. You can use this to pull out data from an environment variable or to perform calculations to generate the needed connection information.

You don't have to update the database configurations manually. If you look at the options of the application generator, you will see that one of the options is named `--database`. This option allows you to choose an adapter from a list of the most used relational databases. You can even run the generator repeatedly: `cd .. && rails new blog --database=mysql`. When you confirm the overwriting of the `config/database.yml` file, your application will be configured for MySQL instead of SQLite. Detailed examples of the common database connections are below.